Source Code Program *Google Collab* Analisis Pola Asosiasi Produk pada Data Transaksi Ritel Menggunakan Algoritma Apriori

Oleh:

Riska Juniarti Anugrah

Keysha Jauzi Althaff

Tiara Ambarwati Rahman

Program Studi Statitsika

Fakultas Matematika dan Ilmu Pengetahuan

Universitas Islam Bandung

2025

DAFTAR ISI

DAFTAR ISI **2**

I. Deskripsi Program **3**

II. Code Program Google Collab **5**

III. Panduan Penggunaan **15**

Source Code Program *Google Collab* Analisis Pola Asosiasi Produk pada Data Transaksi Ritel Menggunakan Algoritma Apriori

I. Deskripsi Program

Program ini menerapkan teknik *data mining* berupa *Association Rule Mining* (ARM) untuk menganalisis pola hubungan antar produk pada data transaksi ritel. *Association Rule Mining* bertujuan untuk menemukan keterkaitan antar item dalam bentuk aturan implikasi *if–then* atau $A \rightarrow B$, yang menunjukkan bahwa kemunculan item pada bagian *antecedent* cenderung diikuti oleh item pada bagian *consequent* dalam transaksi yang sama (Han, Kamber, & Pei, 2012). Teknik ini banyak digunakan dalam analisis data transaksi ritel (*market basket analysis*) untuk memahami perilaku pembelian konsumen. Aturan asosiasi tersebut memberikan informasi bahwa kemunculan item atau kombinasi item pada bagian *antecedent* cenderung diikuti oleh kemunculan item lain pada bagian *consequent* dalam transaksi yang sama. Informasi ini sangat bermanfaat dalam memahami pola perilaku konsumen serta mendukung pengambilan keputusan dalam strategi pemasaran, penataan produk, dan promosi penjualan.

Algoritma yang digunakan dalam program ini adalah Algoritma Apriori, yaitu algoritma dasar dalam *Association Rule Mining* yang diperkenalkan oleh Agrawal dan Srikant (1994). Algoritma Apriori bekerja berdasarkan prinsip *Apriori Property*, yang menyatakan bahwa jika suatu itemset tidak memenuhi kriteria *frequent*, maka seluruh superset dari itemset tersebut juga tidak akan bersifat *frequent*. Prinsip ini digunakan untuk melakukan proses *pruning* sehingga jumlah kandidat itemset yang perlu dievaluasi dapat dikurangi dan efisiensi komputasi dapat ditingkatkan (Agrawal & Srikant, 1994).

Program ini disusun dalam bentuk *source code* menggunakan bahasa pemrograman Python dan dijalankan pada platform Google Colab. Seluruh tahapan analisis dirancang dan disesuaikan dengan studi kasus data transaksi ritel masing-masing kelompok, sehingga implementasi algoritma Apriori yang dilakukan bersifat aplikatif dan relevan dengan permasalahan yang dianalisis.

Tahap awal program adalah melakukan praproses data transaksi, yaitu mengubah data mentah ke dalam format transaksi yang sesuai dengan kebutuhan algoritma Apriori. Setiap transaksi direpresentasikan sebagai kumpulan item yang dibeli secara bersamaan. Selain itu, pada tahap ini juga dilakukan perhitungan statistik deskriptif, seperti jumlah transaksi dan frekuensi kemunculan setiap item, yang digunakan untuk memberikan gambaran awal mengenai karakteristik data sebelum dilakukan analisis lebih lanjut.

Selanjutnya, algoritma Apriori digunakan untuk mencari *frequent itemset*, yaitu kombinasi item yang sering muncul secara bersamaan dalam data transaksi. Penentuan *frequent itemset* didasarkan pada nilai minimum *support* yang telah ditetapkan. Nilai *support* digunakan untuk mengukur seberapa sering suatu kombinasi item muncul dalam keseluruhan transaksi dan dirumuskan sebagai berikut:

$$Support(A \cup B) = \frac{\text{Jumlah transaksi yang mengandung } A \text{ dan } B}{\text{Total transaksi}}$$

Dalam program ini, nilai minimum *support* ditetapkan sebesar 0,01 atau disesuaikan dengan karakteristik data transaksi yang dianalisis. Itemset yang memiliki nilai *support* lebih besar atau sama dengan nilai minimum *support* akan dipertahankan sebagai *frequent itemset*, sedangkan itemset yang tidak memenuhi kriteria tersebut akan dieliminasi melalui proses *pruning*.

Frequent itemset yang telah diperoleh kemudian digunakan untuk membentuk aturan asosiasi dalam bentuk implikasi $A \rightarrow B$, di mana Amerupakan *antecedent* dan Bmerupakan *consequent*. Setiap aturan yang terbentuk selanjutnya dievaluasi untuk mengetahui kekuatan dan keandalannya. Salah satu metrik yang digunakan dalam evaluasi aturan asosiasi adalah *confidence,* yang mengukur tingkat kepercayaan suatu aturan, yaitu peluang munculnya item Bpada suatu transaksi dengan syarat item A telah muncul terlebih dahulu. Nilai *confidence* dirumuskan sebagai berikut:

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$$

Pada tahap *pruning* aturan asosiasi, program ini menetapkan nilai minimum *confidence* sebesar 0,3. Aturan asosiasi yang memiliki nilai *confidence* di bawah ambang batas tersebut akan dieliminasi karena dianggap memiliki tingkat kepercayaan yang rendah. Selain *confidence*, evaluasi akhir terhadap aturan asosiasi dilakukan menggunakan metrik *lift*, yang digunakan untuk menilai kekuatan hubungan antara item Adan Bdengan mempertimbangkan peluang kemunculan item secara independen. Nilai *lift* dirumuskan sebagai berikut:

$$Lift(A \rightarrow B) = \frac{Confidence(A \rightarrow B)}{Support(B)}$$

Nilai *lift* memberikan interpretasi yang lebih mendalam terhadap hubungan antar item. Aturan dengan nilai *lift* lebih besar dari satu ($Lift > 1$) menunjukkan adanya hubungan positif antar item, sedangkan nilai *lift* sama dengan satu menunjukkan bahwa item bersifat independen. Apabila nilai *lift* kurang dari satu, maka hubungan yang terbentuk bersifat negatif. Dalam program ini, aturan asosiasi dengan nilai *lift* lebih besar dari satu dipilih sebagai pola hubungan yang signifikan dan menjadi fokus utama dalam analisis. Seluruh proses analisis, mulai dari praproses data, perhitungan statistik deskriptif, pembentukan *frequent itemset*, hingga evaluasi aturan asosiasi menggunakan *support*, *confidence*, dan *lift*, diimplementasikan secara langsung di dalam *source code* program. Dengan demikian, deskripsi metodologi yang disampaikan sejalan dan konsisten dengan proses komputasi yang dijalankan oleh program.

Melalui program ini, diharapkan dapat diperoleh informasi yang komprehensif mengenai pola pembelian konsumen pada data transaksi ritel. Hasil berupa *frequent itemset* dan aturan asosiasi yang signifikan dapat digunakan sebagai dasar pertimbangan dalam pengambilan keputusan, seperti penyusunan strategi promosi, pengelompokan produk, serta peningkatan efektivitas penjualan. Dengan mengintegrasikan konsep teoritis *Association Rule Mining* dan Algoritma Apriori ke dalam implementasi *source code* yang disesuaikan dengan studi kasus, program ini bersifat aplikatif dan mampu memberikan wawasan yang bermakna terhadap data yang dianalisis.

II. Source Code Program Google Collab

Link *source code*:

https://colab.research.google.com/drive/1keTzFAI_sWUc1iWdN4Imh4ELkupLCWDk?usp=sharing

```
# ====================================
# KONFIGURASI (BIAR OUTPUT BERSIH)
# ====================================
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# INSTALL LIBRARY
# ====================================
#!pip install mlxtend plotly

# ====================================
# IMPORT LIBRARY
# ====================================
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import plotly.express as px
import networkx as nx
import matplotlib.pyplot as plt
import ast

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# LOAD DATASET
# ====================================
from google.colab import drive
drive.mount('/content/drive')

Retail = pd.read_csv(
    "/content/drive/MyDrive/Project Prak Datmin/Retail.csv",
    encoding="latin1",
    low_memory=False
)

print(Retail.head())
print("Jumlah data awal:", Retail.shape[0])

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
# ====================================
# SAMPLING DATA (1000 TRANSAKSI)
# ====================================
Retail = Retail.sample(n=1000, random_state=42)
print("Jumlah data setelah sampling:", Retail.shape[0])

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# Menampilkan 5 data awal hasil sampling
print("\n5 Data Awal Setelah Sampling:")
display(Retail.head())

# Menampilkan 5 data akhir hasil sampling
print("\n5 Data Akhir Setelah Sampling:")
display(Retail.tail())

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# STATISTIK DESKRIPTIF
# ====================================
Retail_sample = Retail.sample(n=1000,
random_state=42).reset_index(drop=True)

print("Jumlah data sampling:", Retail_sample.shape[0])

stat_desc = Retail_sample[['Total_Items', 'Total_Cost']].describe()
display(stat_desc)

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Distribusi Jumlah Item per Transaksi
# ====================================

# Hitung jumlah transaksi untuk setiap Total_Items
items_count = Retail['Total_Items'].value_counts().sort_index()

# Pastikan semua nilai 1-10 muncul
items_count = items_count.reindex(range(1, 11), fill_value=0).reset_index()
items_count.columns = ['Total_Items', 'Jumlah_Transaksi']

fig = px.bar(
    items_count,
    x='Total_Items',
```

```
    y='Jumlah_Transaksi',
    title='Distribusi Jumlah Item per Transaksi',
    text='Jumlah_Transaksi'
)


fig.update_traces(
    marker_color='#4C78A8',
    textposition='outside'
)

fig.update_layout(
    xaxis_title='Jumlah Item per Transaksi',
    yaxis_title='Jumlah Transaksi',
    bargap=0.35,
    uniformtext_minsize=10,
    uniformtext_mode='hide'
)

fig.show()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Distribusi Metode Pembayaran (Pie Chart)
# ====================================
payment_df = Retail['Payment_Method'].value_counts().reset_index()
payment_df.columns = ['Payment_Method', 'Jumlah_Transaksi']

fig = px.pie(
    payment_df,
    names='Payment_Method',
    values='Jumlah_Transaksi',
    title='Distribusi Metode Pembayaran',
    color_discrete_sequence=[
        '#1f3b4d',
        '#4f6d7a',
        '#9db4c0',
        '#dbe9ee'
    ]
)

fig.update_traces(
    textposition='inside',
    textinfo='percent+label',
    marker=dict(line=dict(color='white', width=1))
)
```

```
fig.update_layout(
    font=dict(size=12),
    showlegend=True
)

fig.show()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Distribusi Store Type
# ====================================
store_df = Retail['Store_Type'].value_counts().reset_index()
store_df.columns = ['Store_Type', 'Jumlah_Transaksi']

fig = px.bar(
    store_df,
    x='Store_Type',
    y='Jumlah_Transaksi',
    title="Distribusi Transaksi Berdasarkan Store Type",
    text='Jumlah_Transaksi',
    color_discrete_sequence=['#4C6A92']
)

# posisi & format angka
fig.update_traces(
    textposition='outside',
    texttemplate='%{text:,}'  # pakai pemisah ribuan
)

fig.update_layout(
    xaxis_title="Store Type",
    yaxis_title="Jumlah Transaksi",
    showlegend=False,
    bargap=0.35,
    plot_bgcolor='white'
)

fig.show()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

season_df = Retail['Season'].value_counts().reindex(
    ['Spring', 'Summer', 'Fall', 'Winter']
).reset_index()
```

```
season_df.columns = ['Season', 'Jumlah_Transaksi']

fig = px.bar(
    season_df,
    x='Season',
    y='Jumlah_Transaksi',
    text='Jumlah_Transaksi',
    color_discrete_sequence=['#4E6E81'],
    title='Distribusi Transaksi Berdasarkan Musim'
)

fig.update_traces(textposition='outside')
fig.update_layout(
    xaxis_title='Musim',
    yaxis_title='Jumlah Transaksi',
    bargap=0.35,
    showlegend=False
)

fig.show()
```

PREPROCESSING

```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

#====================================
#  Missing Value
# ====================================
display(Retail.isnull().sum())

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)


# ====================================
#  Duplikasi Data
# ====================================
print("Jumlah data duplikat:", Retail.duplicated().sum())

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Ubah kolom Product dari string ke list (Pembentukan Data Transaksi)
# ====================================
Retail['Product'] = Retail['Product'].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)
```

```
# Bentuk list transaksi
transactions_list = Retail['Product'].tolist()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# TRANSFORMASI ONE-HOT ENCODING
# ====================================
te = TransactionEncoder()
te_ary = te.fit(transactions_list).transform(transactions_list)
df_encoded = pd.DataFrame(te_ary, columns=te.columns_)

# UBAH True/False → 1/0
df_encoded = df_encoded.astype(int)

print("5 baris pertama dari data hasil one-hot encoding:")
display(df_encoded.head())

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# APRIORI FREQUENT ITEMSET
# ====================================
frequent_itemsets = apriori(
    df_encoded,
    min_support=0.001,
    use_colnames=True,
    max_len=2
)

display(frequent_itemsets.sort_values(by='support',
ascending=False).head(10))

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Frequent 1-Itemsets
# ====================================

freq_1 = frequent_itemsets[
    frequent_itemsets['itemsets'].apply(lambda x: len(x) == 1)
].sort_values(by='support', ascending=False)

# Reset index & buat kolom No
```

```
freq_1 = freq_1.reset_index(drop=True)
freq_1.insert(0, 'No', range(1, len(freq_1) + 1))

# Ubah frozenset -> string
freq_1['Itemsets'] = freq_1['itemsets'].apply(lambda x: ', '.join(list(x)))

# Pilih kolom akhir
freq_1_display = freq_1[['No', 'support', 'Itemsets']]

display(freq_1_display.head(20))

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Frequent 2-Itemsets
# ====================================

freq_2 = frequent_itemsets[
    frequent_itemsets['itemsets'].apply(lambda x: len(x) == 2)
].sort_values(by='support', ascending=False)

# Reset index & buat kolom No
freq_2 = freq_2.reset_index(drop=True)
freq_2.insert(0, 'No', range(1, len(freq_2) + 1))

# Ubah frozenset -> string
freq_2['Itemsets'] = freq_2['itemsets'].apply(lambda x: ', '.join(list(x)))

# Pilih kolom akhir
freq_2_display = freq_2[['No', 'support', 'Itemsets']]

display(freq_2_display.head(20))

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# ASSOCIATION RULES
# ====================================
rules = association_rules(
    frequent_itemsets,
    metric="lift",
    min_threshold=0.3
)

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

```
# ====================================
# TAMPILKAN ATURAN ASOSIASI
# ====================================
rules_display = rules[['antecedents', 'consequents', 'support',
'confidence', 'lift']]
print("Aturan Asosiasi yang Ditemukan:")
print(rules_display.sort_values(by='lift', ascending=False).head(10))

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# AMBIL 50 ATURAN TERATAS
# ====================================
top_rules = rules.sort_values(by='lift', ascending=False).head(50).copy()

# Convert frozenset ke string (UNTUK PLOT)
top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ',
'.join(list(x)))
top_rules['consequents'] = top_rules['consequents'].apply(lambda x: ',
'.join(list(x)))

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# Scatter Plot Aturan Asosiasi (FIX)
# ====================================

rules_plot = rules.copy()

# Ubah frozenset ke string
rules_plot['antecedents'] = rules_plot['antecedents'].apply(
    lambda x: ', '.join(list(x))
)
rules_plot['consequents'] = rules_plot['consequents'].apply(
    lambda x: ', '.join(list(x))
)

fig = px.scatter(
    rules_plot,
    x="support",
    y="confidence",
    size="lift",
    color="lift",
    color_continuous_scale="Plasma",
    hover_data=["antecedents", "consequents"],
```

```
    title="Scatter Plot Aturan Asosiasi"
)

fig.show()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

rules_graph = rules.sort_values(by='lift', ascending=False).head(15)

G = nx.DiGraph()

for _, row in rules_graph.iterrows():
    for a in row['antecedents']:
        for c in row['consequents']:
            G.add_edge(a, c, lift=row['lift'])

plt.figure(figsize=(12,9))
pos = nx.spring_layout(G, k=0.8, seed=42)

edges = G.edges(data=True)
edge_lift = [d['lift'] for (_, _, d) in edges]

# =====================
# NODE
# =====================
nx.draw_networkx_nodes(
    G, pos,
    node_size=1500,
    node_color="#8ecae6",
    edgecolors="black"
)

# =====================
# EDGE (SATU WARNA)
# =====================
nx.draw_networkx_edges(
    G, pos,
    width=[l * 1.8 for l in edge_lift],
    edge_color="#6c757d",
    alpha=0.9,
    connectionstyle="arc3,rad=0.15"
)

# =====================
# LABEL
# =====================
nx.draw_networkx_labels(G, pos, font_size=9)
```

```
plt.title("Network Graph Aturan Asosiasi Terkuat (Apriori) - Retail
Dataset")
plt.axis("off")
plt.show()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
# Ambil 10 aturan asosiasi dengan lift tertinggi
top_rules = rules.sort_values(by='lift', ascending=False).head(10)

# Pilih kolom penting
bundling_recommendation = top_rules[
    ['antecedents', 'consequents', 'support', 'confidence', 'lift']
].copy()

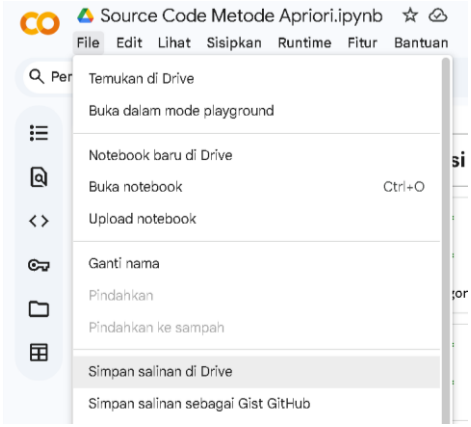
# Ubah frozenset ke string
bundling_recommendation['Produk 1'] =
bundling_recommendation['antecedents'].apply(
    lambda x: ', '.join(list(x))
)
bundling_recommendation['Produk 2'] =
bundling_recommendation['consequents'].apply(
    lambda x: ', '.join(list(x))
)

bundling_recommendation = bundling_recommendation[
    ['Produk 1', 'Produk 2', 'confidence', 'lift']
]

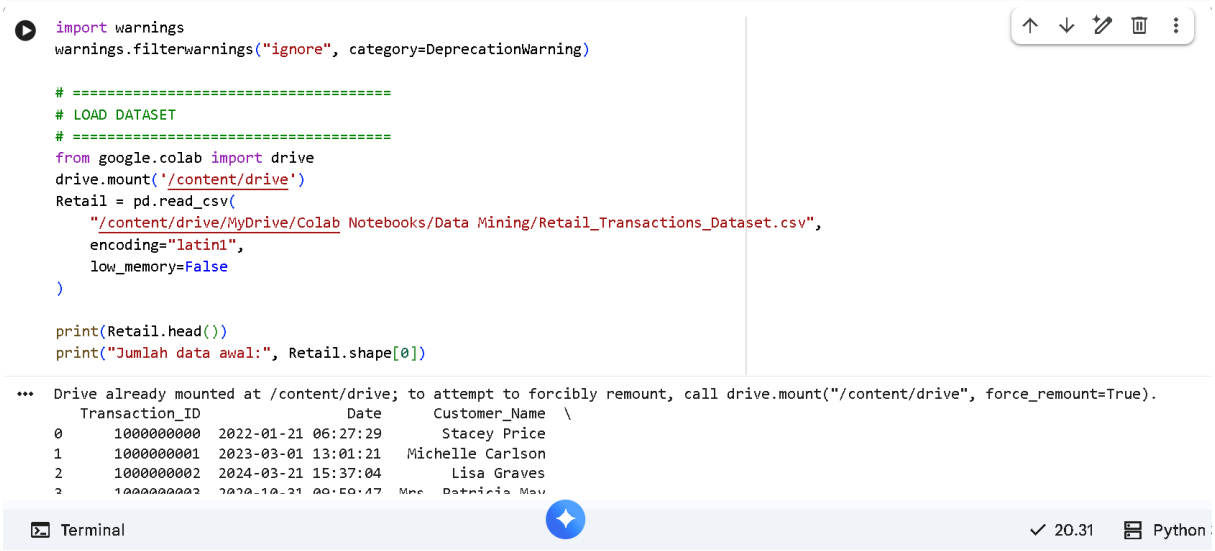
display(bundling_recommendation)
```

III. Panduan Penggunaan

1. Buka tautan Google Colaboratory yang telah disediakan
   Tautan:
   https://colab.research.google.com/drive/1keTzFAI_sWUc1iWdN4Imh4ELkupLCWDk?usp=sharing
2. Buat salinan notebook dengan memilih menu File lalu *Save a copy in Drive* agar notebook dapat diedit.



3. Jalankan sel kode untuk mengimpor *library* yang dibutuhkan menggunakan tombol Shift + Enter.
4. Hubungkan Google Collab dengan Google Drive untuk mengakses dataset. Pastikan file dataset tersedia pada folder Google Drive yang terhubung.



```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# ====================================
# LOAD DATASET
# ====================================
from google.colab import drive
drive.mount('/content/drive')
Retail = pd.read_csv(
    "/content/drive/MyDrive/Colab Notebooks/Data Mining/Retail_Transactions_Dataset.csv",
    encoding="latin1",
    low_memory=False
)

print(Retail.head())
print("Jumlah data awal:", Retail.shape[0])
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
   Transaction_ID                 Date      Customer_Name  \
0      1000000000  2022-01-21 06:27:29       Stacey Price
1      1000000001  2023-03-01 13:01:21   Michelle Carlson
2      1000000002  2024-03-21 15:37:04        Lisa Graves
```

Terminal 20.31 Python

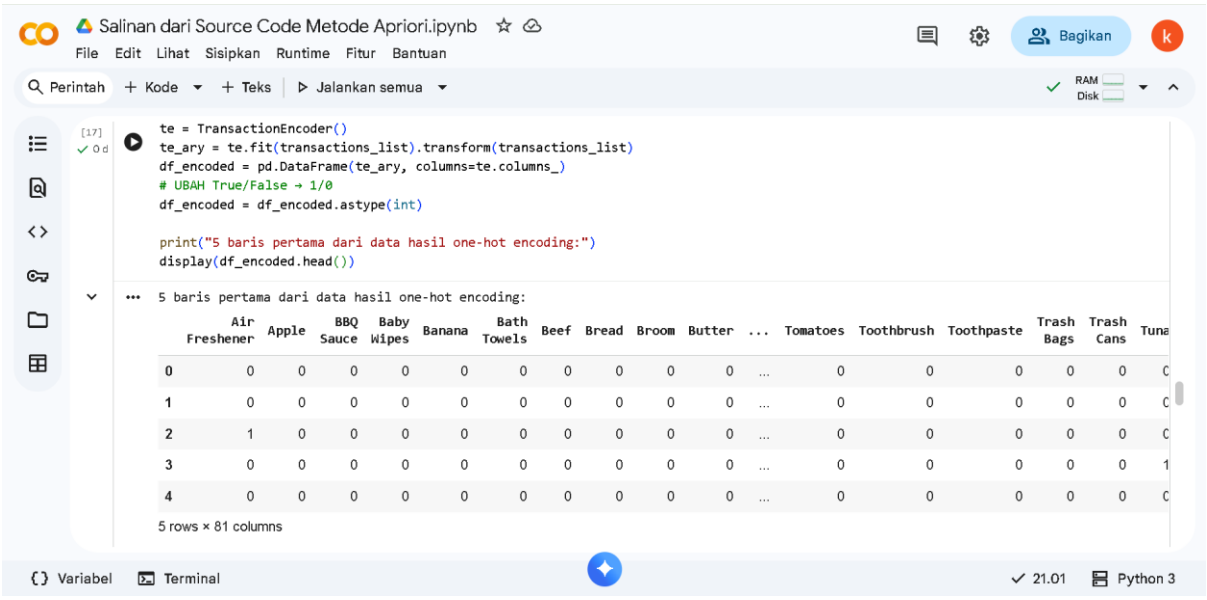
Untuk menggunakan source code Algoritma Apriori, diperlukan data transaksi yang memuat informasi item yang muncul dalam setiap transaksi. Data transaksi tersebut diimpor ke dalam sistem melalui Google Colaboratory dan dibaca untuk memastikan data berhasil dimuat dengan benar. Pada contoh penerapan ini, digunakan data transaksi ritel, yaitu data transaksi penjualan yang berisi daftar produk yang dibeli oleh pelanggan dalam setiap transaksi. Data transaksi yang relatif besar, dapat dilakukan pembatasan jumlah data sebagai penyesuaian untuk efisiensi komputasi.

Berikut merupakan datanya:

No	Transaction ID	Date	Customer Name	Product	Total Items	Total Cost	Payment Method	City	Store Type	Discount Applied	Customer Category	Season	Promotion
1	1000987231	5/23/2023 18:07	Jacqueline Maynard	Pancake Mix, Coffee, Ice Cream	3	16.63	Credit Card	Houston	Specialty Store	TRUE	Middle-Aged	Winter	–
2	1000079954	3/10/2022 16:47	David White	Cleaning Spray	5	7.46	Debit Card	Miami	Supermarket	FALSE	Middle-Aged	Fall	BOGO
3	1000567130	7/22/2022 17:02	Dr. Thomas Edwards	Salmon, Garden Hose, Deodorant, Air Freshener	9	52.27	Cash	Boston	Warehouse Club	TRUE	Teenager	Fall	BOGO
4	1000500891	4/21/2022 15:14	Kristin Collins	Power Strips, Tuna	5	25.44	Cash	Los Angeles	Convenience Store	FALSE	Homemaker	Spring	–
5	1000055399	5/3/2023 5:26	Robert Finley	Cereal, Cheese, Yogurt, Sponges, Chicken	6	86.05	Credit Card	Atlanta	Pharmacy	FALSE	Professional	Fall	BOGO
…	…	…	…	…	…	…	…	…	…	…	…	…	…
996	1000914398	6/10/2021 18:32	Eric Evans	Trash Cans, Air Freshener, Hand Sanitizer	6	16.59	Credit Card	New York	Supermarket	FALSE	Senior Citizen	Spring	Discount
997	1000189053	12/28/2022 19:20	Haley Spears	Laundry Detergent, Shower Gel, Baby	10	38.75	Credit Card	San Francisco	Pharmacy	TRUE	Teenager	Spring	Discount

No	Transaction ID	Date	Customer Name	Product	Total Items	Total Cost	Payment Method	City	Store Type	Discount Applied	Customer Category	Season	Promotion
				Wipes									
998	1000047977	11/1/2022 8:21	Erika Mcculough	Broom, Dustpan, Trash Cans, Garden Hose	8	79.49	Mobile Payment	Boston	Warehouse Club	TRUE	Homemaker	Winter	–
999	1000600785	6/1/2023 8:39	Garrett Mcgee	Olive Oil	4	31.93	Mobile Payment	San Francisco	Convenience Store	TRUE	Teenager	Spring	–
1000	1000067400	6/7/2020 19:20	Gordon Hudson	Beef	1	88	Cash	Atlanta	Specialty Store	FALSE	Senior Citizen	Winter	Discount

5. Setelah data berhasil diimpor, dilakukan pemeriksaan dan penyesuaian data dasar untuk memastikan format data sesuai dan siap dianalisis. Selanjutnya, data transaksi ditransformasikan ke dalam bentuk *one-hot encoding* sehingga setiap item direpresentasikan sebagai variabel biner sesuai dengan kebutuhan Algoritma Apriori.



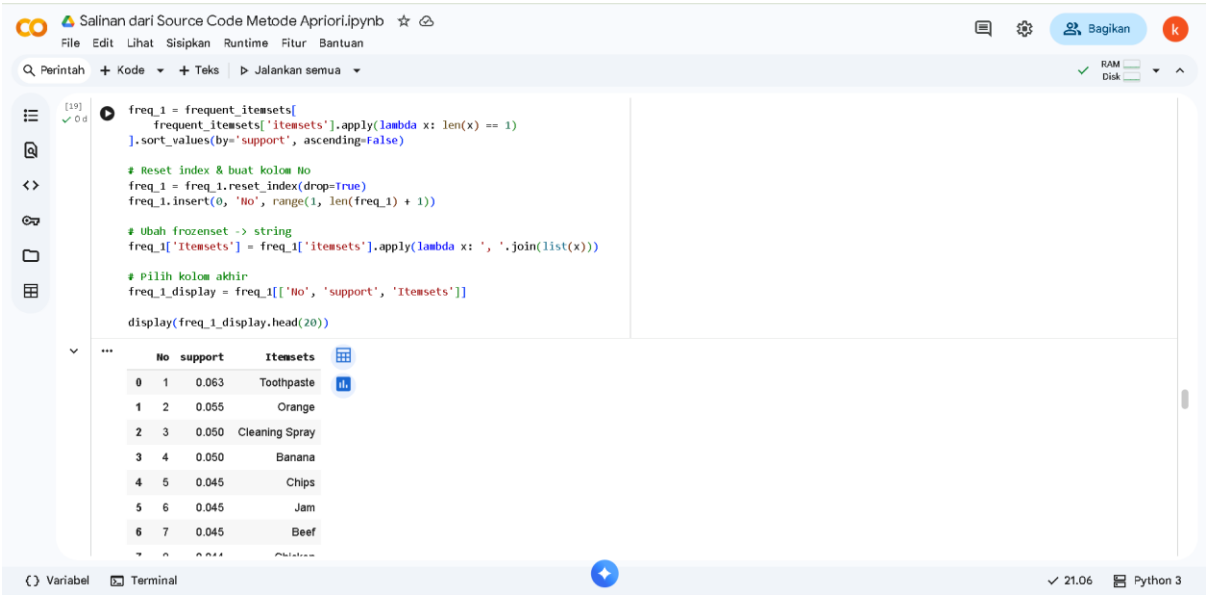
6. Algoritma Apriori kemudian dijalankan untuk menghasilkan *frequent itemsets* dan aturan asosiasi berdasarkan parameter yang telah ditentukan. Algoritma Apriori menghasilkan *frequent itemset*s yang kemudian dikelompokkan menjadi *frequent 1-itemsets* dan *frequent 2-itemsets* untuk mengidentifikasi produk individual dan pasangan produk yang sering muncul dalam transaksi.

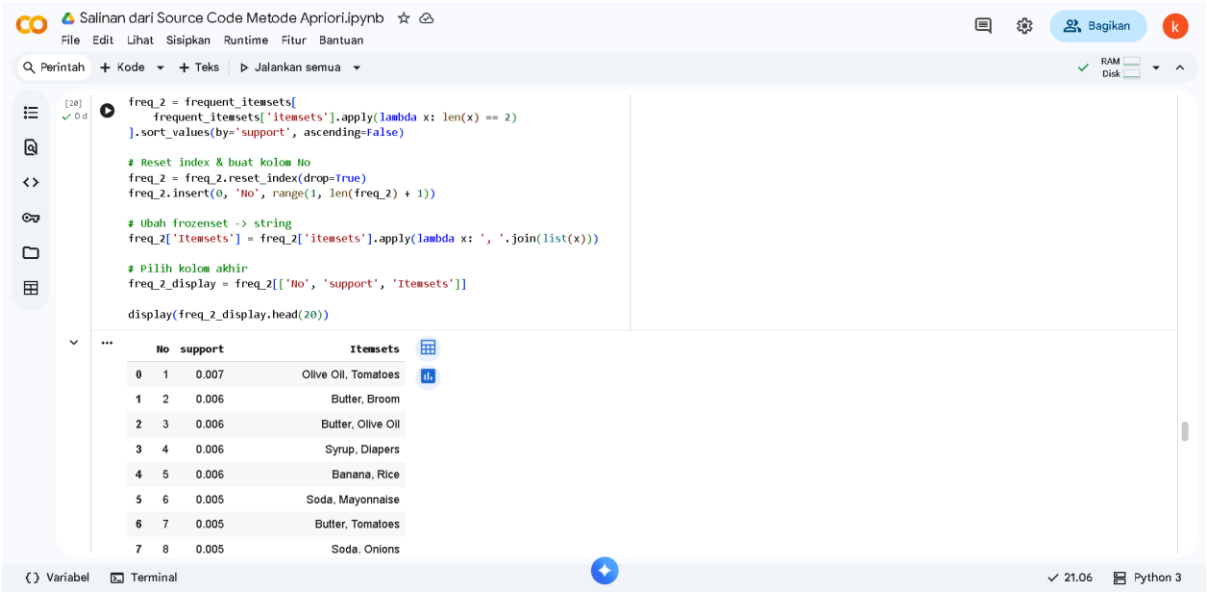
Berikut merupakan hasil aturan asosiasi yang diperoleh dari penerapan Algoritma Apriori:

```
frequent_itemsets = apriori(
    df_encoded,
    min_support=0.001,
    use_colnames=True,
    max_len=2
)

display(frequent_itemsets.sort_values(by='support', ascending=False).head(10))
```

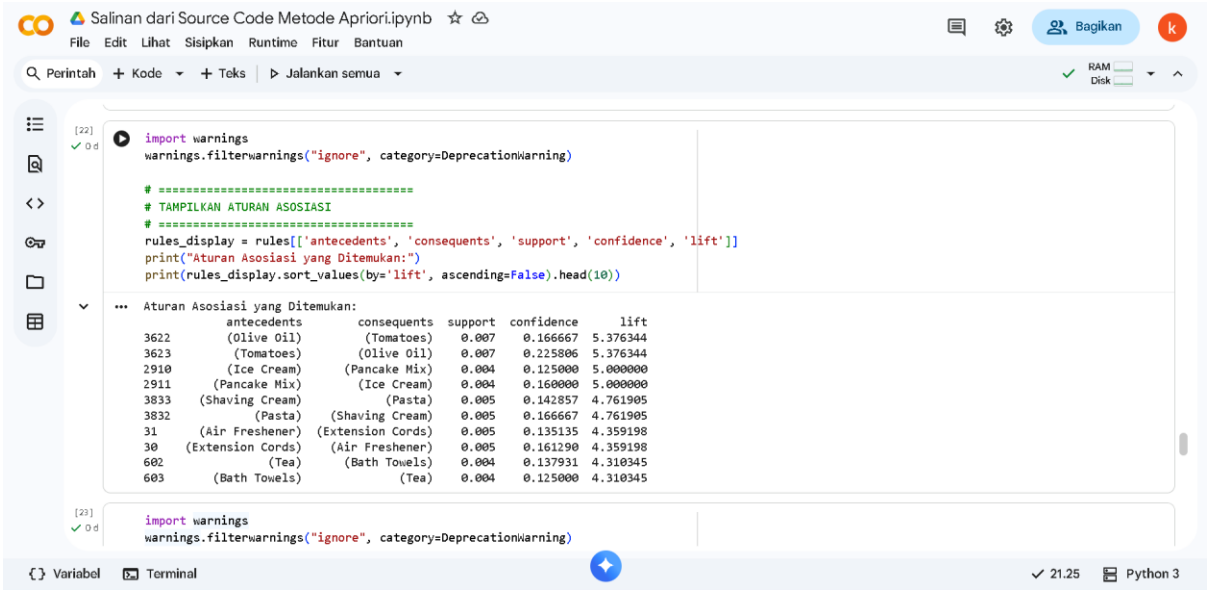
	support	itemsets
73	0.063	(Toothpaste)
47	0.055	(Orange)
18	0.050	(Cleaning Spray)
4	0.050	(Banana)
36	0.045	(Jam)
6	0.045	(Beef)
16	0.045	(Chips)
17	0.044	(Cleaning Rags)
15	0.044	(Chicken)
29	0.043	(Hair Gel)





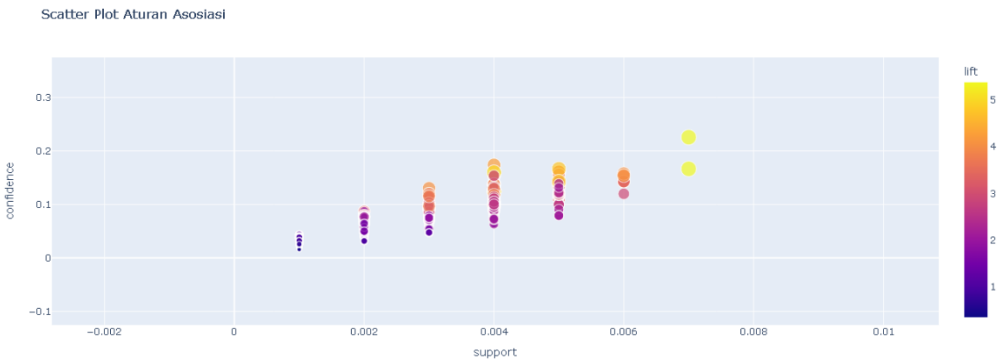
7. Pembentukkan Aturan Asosiasi

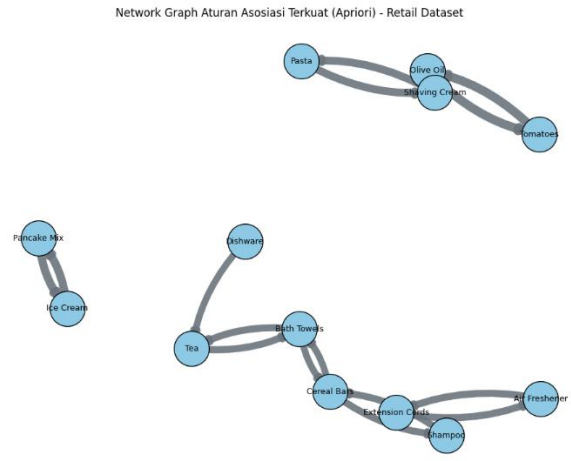
Aturan asosiasi dibentuk dari *frequent itemsets* dengan kriteria, Metric: *lift* dan Minimum *threshold*: 0.3



8. Visualisasi Aturan Asosiasi

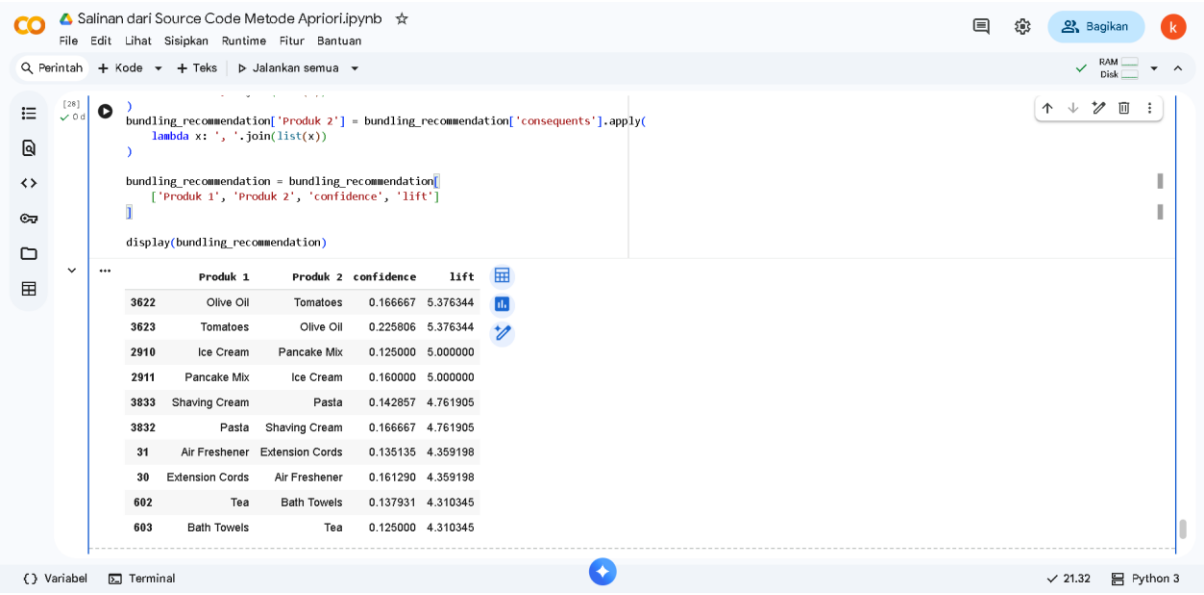
Visualisasi dilakukan untuk mempermudah interpretasi pola asosiasi. Aturan asosiasi diurutkan berdasarkan nilai *lift* dan diambil 50 aturan teratas sebagai representasi pola asosiasi terkuat.





9. Rekomendasi Bundling Produk

Diambil 10 aturan asosiasi dengan nilai *lift* tertinggi sebagai dasar rekomendasi *bundling* produk.



	Produk 1	Produk 2	confidence	lift
3622	Olive Oil	Tomatoes	0.166667	5.376344
3623	Tomatoes	Olive Oil	0.225806	5.376344
2910	Ice Cream	Pancake Mix	0.125000	5.000000
2911	Pancake Mix	Ice Cream	0.160000	5.000000
3833	Shaving Cream	Pasta	0.142857	4.761905
3832	Pasta	Shaving Cream	0.166667	4.761905
31	Air Freshener	Extension Cords	0.135135	4.359198
30	Extension Cords	Air Freshener	0.161290	4.359198
602	Tea	Bath Towels	0.137931	4.310345
603	Bath Towels	Tea	0.125000	4.310345

Berdasarkan hasil penerapan algoritma Apriori, diperoleh sejumlah aturan asosiasi yang menunjukkan keterkaitan antar produk dalam data transaksi. Aturan dengan nilai *support, confidence*, dan *lift* yang tinggi mengindikasikan pasangan produk yang sering dibeli secara bersamaan. Hasil ini dapat dimanfaatkan sebagai dasar dalam penyusunan rekomendasi bundling produk atau strategi penataan produk.